



FOR SOFTWARE ENGINEERS: WHAT THE WORK IS NOW

While The AI Writes The Code

The four jobs that are left

Four hundred lines in ninety seconds. One of these four, skipped, makes the other three theatre.

The comfortable version of this page would say that AI does the boring typing while experienced engineers keep doing the important thinking. That is the version a smart sceptic calls cope, and they would be right.

Implementation genuinely was part of the job. The claim that survives is narrower: implementation is getting cheaper, accountability is not going anywhere, and judgement is now the scarce resource.

Every figure on this sheet was read at the primary source in September 2026 and is quoted with its limitations attached. One of the studies has since been labelled historical by the team that ran it, and this sheet says so where it quotes it. A number without its caveat is not a stronger claim, it is a less honest one.

In scope

- The four responsibilities, in the order they happen
- What independent evidence looks like, matched to the claim
- Three studies, each quoted with its limitation attached
- The failure mode that undoes all four in one sentence
- The strongest argument against this whole page

Out of scope

- Which assistant to use. This is about the job, not the tool
- Prompt technique, which the tools are absorbing anyway
- Any claim that these four are uniquely human
- Any number nobody measured

Read section four first if you only read one. The other three are what make it possible, but it is the one that decides whether any of it counted.



01 Where a developer workday actually goes, IEEE TSE 2019

Reading or writing code	15%, 84 minutes
Meetings	15%, 85 minutes
Debugging or fixing bugs	14%, 74 minutes
Emails	10%, 53 minutes
Testing	8%, 41 minutes
Helping or mentoring others	5%, 26 minutes
Longest uninterrupted stretch on code	47.3 minutes, on average

5,971 self reports from professional developers. Self reported, at one very large company. It shows the job was already bigger than the part being automated. It does NOT prove typing was merely a bottleneck.

03 One. Choose the smallest correct change

THE REASON THIS IS FIRST IS ECONOMIC. An unnecessary feature used to cost an estimate, a build, a test and a justification for three days. That friction was not efficient, but some bad ideas died of exhaustion. It is gone.

A REAL EXAMPLE, typed at five in the afternoon: "add retries to the payment call, add a cache on the order read, and add a fallback table in case the provider is down." Every word of that is reasonable. Twenty minutes later it is six hundred lines of technically correct liability.

Retries with no idempotency key, so the payment goes twice. A cache with no invalidation story, so a refunded order reads as paid. A fallback table that is now a second source of truth nobody will reconcile.

IT ALSO MEANS THE SHAPE YOU HAND OVER. Not "implement subscriptions". The state model, then the migration, then the API, then the billing integration, then the backfill, each independently deployable and observable. Agents are dramatically better inside narrow boundaries with fast feedback. Setting those boundaries is engineering orchestration, not prompting.

A retry with no idempotency key, implemented exactly as asked, records two charges and bills the customer 9998 cents instead of 4999. The same retry with a key derived from the order records one.

Run on the machine that built this sheet, not quoted

A cache with no invalidation story shows a refunded order as PAID while the database says REFUNDED.

Run on the machine that built this sheet, not quoted

04 Two. Define correctness before the implementation exists

Write acceptance criteria after you have seen the code and you will write criteria the code already passes. Everybody does. This is the cheapest discipline on the page and the one skipped most often, because when the code arrives in ninety seconds, spending ten minutes first feels absurd.

A HANDOVER WITH CORRECTNESS FIRST names the behaviour as OBSERVABLE OUTCOMES, names the cases that must FAIL, and forbids changes outside the named files. The must-fail half is the half everyone leaves out, and it is the half that becomes the incident.

AND THE PART THAT IS IN NO TICKET: the invariants. A payment is charged at most once. A refund never exceeds the original amount. An audit row is never deleted. There is no file called invariants.txt. They live in the heads of people who have been on call, and nothing downstream can respect what nobody stated.

Acceptance criteria written after the implementation exists are criteria the implementation already satisfies.

Operational practice, not vendor documentation

05 Three. Reason about consequences outside the diff

A diff can be entirely correct and still make the system worse, and this is the failure that reads as competence until three in the morning. The change is small, the tests pass, the review is clean, and it added a synchronous call to a service that is 99.5 percent available on a path that was previously local.

THE CLEAREST VERSION IS ANYTHING IRREVERSIBLE. A migration that drops a column is correct, tested and reviewed, and it is the one change in the release that cannot be rolled back by redeploying the previous version. Nothing in the diff says so. It is a property of the change in the WORLD, not of the change in the file.

The practical question, asked of every release: which of these can I undo by redeploying, and which one costs me a restore from backup and the writes since?

Redeploying the previous version undoes a wrong constant, a bad query plan and a new synchronous dependency. It does not undo a dropped column.

Operational practice, not vendor documentation

06 Four. Verify against independent reality

HERE IS THE TEST. Suppose you did everything above perfectly. You cut the scope, wrote the criteria first, stated the invariants, thought about the blast radius, and read the diff line by line. Then you asked: are we done? And it said: yes, all requirements are satisfied and the implementation correctly preserves idempotency.

If you accept that sentence as your evidence, every discipline above it just collapsed. The thing that produced the work became the judge of the work.

THE LINE WORTH KEEPING: never let the system that produced the work be your only source of evidence that the work is correct.

That is stronger than "read the diff", and notice why it survives better models. Reading the diff is a TECHNIQUE for obtaining confidence, and techniques get replaced. Independent evidence is a principle, and it still makes sense no matter how good the thing writing the code becomes.

Tests written after the fix, by the thing that wrote the fix, are not independent evidence.

Operational practice, not vendor documentation

07 Matching the evidence to the claim is the skill

IT SAYS THE ENDPOINT IS IDEMPOTENT: send the request twice and read the row count.

IT SAYS THE MIGRATION IS SAFE: run it against a restored copy of production, and time it.

IT SAYS PERFORMANCE IS FINE: load it, and look at p99 rather than the average.

IT SAYS THE BUG IS FIXED: reproduce the original failure FIRST, watch it fail, then watch it pass.

IT SAYS THE TESTS PASS: whose tests, written when, by what?

Independent means the evidence did not come from the thing being evaluated. Everything else is a detail of which instrument fits which claim.

`Operational practice, not vendor documentation`

08 And if you think you would notice

A RANDOMISED TRIAL RUN BY METR took 16 experienced open source developers and 246 real tasks in their own repositories. With AI tools allowed they were 19 percent SLOWER. They predicted a 24 percent speedup beforehand, and estimated a 20 percent speedup afterwards, having just been slowed down.

BE FAIR ABOUT IT: METR now labels that result historical, because it used early 2025 tools. The finding being relied on here is not the 19 percent. It is the gap between what was measured and what was felt.

FOR BALANCE, a randomised trial run by GITHUB on 202 developers with at least five years of experience, reviewed blind in 1,293 reviews, found participants with Copilot were 53.2 percent more likely to pass all ten unit tests. That is GitHub studying its own product, on one bounded API building task.

Both are evidence. Neither of them is your codebase, which is the reason section four exists at all.

16 developers, 246 real tasks, 19 percent slower with AI tools allowed, against a forecast of 24 percent faster and a retrospective estimate of 20 percent faster. Now labelled historical by METR.

`METR, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity"`

202 developers, 1,293 blind reviews, 53.2 percent greater likelihood of passing all ten unit tests, on one bounded task, in a study run by GitHub.

`GitHub, "Does GitHub Copilot improve code quality? Here is what the data says"`

Reading or writing code is 15 percent of the workday, 84 minutes, across 5,971 self reports.

`Meyer et al., "Today was a Good Day: The Daily Life of Software Developers", IEEE TSE 2019`

09 The best argument against this sheet

THE OBJECTION, stated fairly: you have taken the shrinking remainder of the job and declared that remainder to have always been the real job. Today it writes the functions. Tomorrow it writes the acceptance criteria, finds the invariants, analyses the blast radius and runs the verification. You are moving the goalposts every time the model gains a capability.

THE ANSWER THAT IS NOT AVAILABLE: "AI will never understand the business like a human", or "humans will always be needed". Neither can be substantiated, and offering them concedes the argument.

THE HONEST ANSWER: yes, some of this will be automated too. These four are not claimed to be uniquely human. The claim is that production is getting cheaper faster than accountability is disappearing, and somebody still has to decide what evidence is sufficient and what risk is acceptable.

If a system can one day own that, including responsibility for the outcome, then the job has changed far more profoundly than this page describes. Saying so out loud is what makes the rest of it credible.

Describing where the bottleneck appears to be moving is a defensible claim. Defending employment is not.

`Operational practice, not vendor documentation`

Before you approve what the agent just wrote

1. Is this the smallest change that is actually correct?
2. Did I write the acceptance criteria BEFORE seeing the code?
3. Did I write the cases that must FAIL, not only the ones that must pass?
4. Which two or three invariants must survive this, and did I say them out loud?
5. What does this change that is not in the diff?
6. Can this be undone by redeploying the previous version?
7. What is my evidence, and did it come from the thing that wrote the code?
8. If it claims idempotency, did I send the request twice and count the rows?
9. Were the tests written before the fix, or after it, and by what?
10. Am I deciding this from a measurement, or from how it feels?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of the four your team actually does.

Study findings are attributed to their authors and were read at the primary source rather than from a summary. Product and company names appear only to identify what was studied. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-15 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.